Spack and PyTorch Development Workflows

Rohit Goswami · 2022-04-24 · ramblings · pytorch

> *`pytorch` local development environments without tears, root access or `spack dev-build`*

Background

Build systems are a bit of a nightmare. I spend most of my time SSH'ed into more powerful CPU machines, and sometimes on machines with more exotic compute devices. `micromamba` is typically my poison of choice where `nix` is not an option.

However, `micromamba` doesn't allow a whole lot in the way of setting up environments which do not use packages already on `conda-forge`. Spack fills a nice niche for such situations[^fn:1]. Additionally, it can be coerced into use for local development and the same cannot easily be said of `conda` based workflows.

Humble Origins

We'll start from scratch, grabbing `spack` and bootstrapping `clingo` for its dependency resolution as per the documentation.

```
git clone -c feature.manyFiles=true https://github.com/spack/spack.git
. spack/share/spack/setup-env.sh # Assumes zsh || bash
spack spec zlib # Or anything really
spack --bootstrap find # Should have clingo
```

Environments and Purity

Unlike `nix`, `spack` does not offer strong guarantees of purity. There are primarily two approaches to using environments in `spack`, somewhat analogously to the `conda` environment logic.

Named Environments

These are essentially equivalent to `conda` environments, and can be activated and deactivated at will. However, the folder structure in this case, along with the dependencies are localized within `$SPACK_HOME/var/spack/environments/$ENV_NAME`

Anonymous Environments

These can be setup inside any directory, and can be activated but not deactivated (`despacktivate` will not work). These are useful for development environments.

We will use both kinds of environments. Additionally, `spack` supports multiple `variants` which can be queried by `spack info $PKG_NAME`. These are used to support various build configurations while providing a unified interface build through `spack install`.

Basic Anonymous Environments

For starters we will need to setup a development environment.

```
mkdir spackPyTorch
cd spackPyTorch
spack env create -d .
```

To ensure that the dependencies are resolved consistently, the `concretization` option to needs to be set to `together`. We will start by adding some packages.

```
spack -e . add py-ipython
spack -e . config edit # Make changes
```

For ease of manual customization, it is best to lightly edit the `spack.yaml` file to have each dependency on its own line. At this point a bare-minimum near-empty environment is ready.

```yaml { hl_lines=["5"] } spack: specs: - py-ipython view: true concretization: together
```

```
To ensure dependencies are resolved simultaneously, `concretize` is set to
`together`.

```bash
spack -e . concretize # doesn't install
spack -e . install
spack env activate . # or spacktivate .
# Can also use the fully qualified path instead of .
```

Recall that changes are propagated via:

```
spack -e . concretize --force
spack -e . install
```

This establishes baseline environments, but precludes setting up development workflows. Most packages can be included as described above, with the exception of compiler toolchains.

Compiler Setups

Setting up compiler toolchains like a `fortran` compiler (perhaps for `openblas`) can take a bit more effort. Although this discussion will focus on obtaining a `fortran` compiler it is equally applicable to updating a version. For `spack`, unlike many system package managers, `gcc` will install the `C`, `C++` and `Fortran` toolchains. Thus:

```
# outside the spack environment
spack install gcc@11.2.0 # will take a while
```

Now we need to let `spack` register this compiler.

```
spack compiler find $(spack location -i gcc@11.2.0)
spack compilers # should now list GCC 11.2.0
```

Finally we can edit our `spack.yml` environment to reflect the new compiler toolchain.

```yaml { hl_lines=["2-3"] } spack: definitions: - compilers: [gcc@11.2.0] specs: - py-ipython view: true concretization: together
```
```

```
One of the nicer parts of `spack` being an HPC environment management tool is
that there is first-class support for proprietary compiler toolchains like Intel
and families of compilers can also be specified with the `%intel` syntax as
well. Much [more fine-tuning](https://spack-
tutorial.readthedocs.io/en/latest/tutorial_configuration.html#yaml-format) is also possible including
registering system
compilers if required.

## PyTorch Local Development {#pytorch-local-development}

One of the caveats of local development with `spack` is that the base URL needs
to be updated from within the local copy of `spack`. This means editing:

```bash
vim $SPACK_ROOT/var/spack/repos/builtin/packages/py-torch/package.py # edit
```

The patch is minimally complicated, for a fork and a working branch like `npeye` it would look like this diff:

```
diff --git a/var/spack/repos/builtin/packages/py-torch/package.py b/var/spack/repos/builtin/packages/py-
torch/package.py
index 8190e01102..c276009d10 100644
--- a/var/spack/repos/builtin/packages/py-torch/package.py
+++ b/var/spack/repos/builtin/packages/py-torch/package.py
@@ -14,7 +14,7 @@ class PyTorch(PythonPackage, CudaPackage):
     with strong GPU acceleration."""

     homepage = "https://pytorch.org/"
-    git      = "https://github.com/pytorch/pytorch.git"
+    git      = "https://github.com/HaoZeke/pytorch.git"

     maintainers = ['adamjstewart']

@@ -22,6 +22,7 @@ class PyTorch(PythonPackage, CudaPackage):
     # core libraries to ensure that the package was successfully installed.
     import_modules = ['torch', 'torch.autograd', 'torch.nn', 'torch.utils']

+    version('npeye', branch='npeye', submodules=True)
     version('master', branch='master', submodules=True)
     version('1.11.0', tag='v1.11.0', submodules=True)
     version('1.10.2', tag='v1.10.2', submodules=True)
@@ -348,7 +349,8 @@ def enable_or_disable(variant, keyword='USE', var=None, newer=False):
         elif '~onnx_ml' in self.spec:
             env.set('ONNX_ML', 'OFF')

-        if not self.spec.satisfies('@master'):
+        if not (self.spec.satisfies('@master') or
+                self.spec.satisfies('@npeye')):
             env.set('PYTORCH_BUILD_VERSION', self.version)
             env.set('PYTORCH_BUILD_NUMBER', 0)
```

Applying such a patch is straightforward.

```
cd $SPACK_ROOT
# Assuming it is named pytorchspack.diff
git apply pytorchspack.diff
```

This modified environment can now be enabled for use with the appropriate variants (details found with `spack info py-torch`). However, there is one rather important variant missing for local development, `DEBUG`. The application of this patch will rectify this until an upstream PR is merged.
```

```
diff --git a/var/spack/repos/builtin/packages/py-torch/package.py b/var/spack/repos/builtin/packages/py-
torch/package.py
index 8190e01102..d7b68ae4bd 100644
--- a/var/spack/repos/builtin/packages/py-torch/package.py
+++ b/var/spack/repos/builtin/packages/py-torch/package.py
@@ -49,6 +49,7 @@ class PyTorch(PythonPackage, CudaPackage):

     # All options are defined in CMakeLists.txt.
     # Some are listed in setup.py, but not all.
+    variant('debug', default=False, description="Build with debugging support")
     variant('caffe2', default=True, description='Build Caffe2', when='@1.7:')
     variant('test', default=False, description='Build C++ test binaries')
     variant('cuda', default=not is_darwin, description='Use CUDA')
@@ -343,6 +344,12 @@ def enable_or_disable(variant, keyword='USE', var=None, newer=False):
         enable_or_disable('gloo', newer=True)
         enable_or_disable('tensorpipe')

+        if '+debug' in self.spec:
+            env.set('DEBUG', 1)
+        elif '-debug' or '~debug' in self.spec:
+            env.set('DEBUG', '0')
+
+
         if '+onnx_ml' in self.spec:
             env.set('ONNX_ML', 'ON')
         elif '~onnx_ml' in self.spec:
```

All together now the variants can be used in conjunction with the development branch. To focus on the `pytorch` workflow, we will unpin `python@3.10` and add `ipython` instead.

CPU

Normally, for a CPU build we would setup something like the following:

```
DEBUG=1 USE_DISTRIBUTED=0 USE_MKLDNN=0 USE_CUDA=0 BUILD_TEST=0 USE_FBGEMM=0 USE_NNPACK=0 USE_QNNPACK=0
USE_XNNPACK=0 BUILD_CAFFE2=0 USE_KINETO=0 python setup.py develop
```

The rationale behind the build flags can be found in the upstream contributing guide. In an appropriately defined environment. For us, this translates to (with the patch added for `debug`):

```
spacktivate .
# CPU only, disable cuda
# Also disable a bunch of optionals
spack add py-torch@npeye -cuda -fbgemm -nnpack -mkldnn -test -qnnpack +debug
spack concretize -f
```

However, we need to also register this package for development (with the branch setup previously), which is accomplished by:

```
spack develop py-torch@npeye -cuda -fbgemm -nnpack -mkldnn -test -qnnpack +debug
```

This generates a sub-directory `py-torch` with the right branch checked out, along with the dependencies needed for the build. Additionally, the `python` version is also localized to the `spack` installation spec.

```
spack install # concretizes and installs
```

If additional dependencies are required for testing or other purposes, they are easily obtained. After making changes `spack install` will rebuild with the appropriate flags.

CUDA Setup

The good news is that updating the build system to use CUDA is very straightforward. While changing variants, it is occasionally necessary to forcibly clear out the cache.

```
cd py-torch
rm -rf build
python setup.py clean
# More extreme cases
git submodule deinit -f .
git clean -xdf
python setup.py clean
git submodule update --init --recursive --jobs 0
python setup.py develop
```

We will require the CUDA version to install the appropriate tool-chain.

```
export NVCC_CUDA_VERSION=$(nvidia-smi -q | awk -F': ' '/CUDA Version/ {print $2}')
spack add cuda@$NVCC_CUDA_VERSION
spack concretize --force
spack install
```

One caveat of CUDA installations (which cannot be dealt with here) is that `spack` needs to have read/write access to `/tmp/cuda-installer.log` because of a ridiculous upstream bug.

Finally we update the `spack.yaml` to reflect our new changes:

```
spack:
  definitions:
  - compilers: [gcc@11.2.0]
    # add package specs to the `specs` list
  specs:
  - py-ipython
  - cuda@11.3
  - py-torch@npeye+cuda+debug~fbgemm~mkldnn~nnpack~qnnpack~test
  view: true
  concretization: together
  develop:
    py-torch:
      spec: py-torch@npeye+cuda+debug~fbgemm~mkldnn~nnpack~qnnpack~test
```

Which now works smoothly with `spack install`.

Baseline Environment

Finally, to test changes against the main branch upstream, it is useful to define an environment for the same. This can be named since it allows for better usage semantics with `deactivate`.

```
spack env create pytorchBaseline
spack -e pytorchBaseline add py-torch@master -cuda -fbgemm -nnpack -mkldnn -test -qnnpack
spack -e pytorchBaseline add py-ipython
spack -e pytorchBaseline config edit
# Add compilers, concretization
spacktivate pytorchBaseline
spack concretize --force
spack install
# Do tests, compare
# ...
# Wrap up and deactivate
despacktivate pytorchBaseline
```

Conclusions

Dependency management is always painful. CUDA management doubly so. Better workflows with spack dev-build are available for some packages, like KOKKOS, but `spack dev-build` doesn't work yet for `pytorch`, and also appears to be removed from the present set of tutorials. Personally, I'd still prefer `nix`, but where `micromamba` falls short in terms of source builds, `spack` is a good alternative, if one has the resources to rebuild everything needed.